Sistema Distribuído de Transações Financeiras

O que é e o que eu fiz até agora

Repositório: saga-financial-transactions-api

1. O Padrão Saga

1.1 Definição

O padrão Saga é um padrão de projeto para arquiteturas distribuídas utilizado para manter a consistência de dados entre múltiplos serviços, cada um com seu próprio banco de dados, sem depender de uma transação ACID única e distribuída (o chamado "two-phase commit"). Em arquiteturas de microsserviços, cada serviço é responsável por sua própria persistência, de modo que uma operação de negócio que envolva mais de um serviço precisa ser dividida em uma sequência de transações locais.

Uma Saga é, portanto, uma sequência de transações locais em que cada transação atualiza o banco de dados de um único serviço e, em seguida, publica um evento ou mensagem que aciona a próxima transação local da sequência. Caso uma dessas transações falhe, a Saga executa uma série de transações compensatórias, que desfazem o impacto das transações locais já realizadas anteriormente na sequência.

1.2 Coreografia vs. Orquestração

Existem duas formas principais de implementar o padrão Saga:

- **Saga Coreografada:** não existe um coordenador central. Cada serviço publica eventos de domínio e os demais serviços interessados reagem a esses eventos de forma independente, decidindo por si próprios qual ação tomar (incluindo compensações).
- **Saga Orquestrada:** existe um serviço central (o orquestrador) responsável por comandar explicitamente cada passo da transação distribuída, enviando comandos aos demais serviços e decidindo, a partir das respostas recebidas, se o fluxo deve prosseguir ou se transações compensatórias devem ser disparadas.

1.3 Transações Compensatórias

A característica que define uma Saga completa — e a diferencia de um simples pipeline de eventos — é a existência de transações compensatórias: ações capazes de reverter o efeito de um passo já concluído quando um passo posterior da sequência falha. Sem esse mecanismo, o sistema não tem como garantir a consistência de dados no cenário de falha parcial, que é justamente o problema que o padrão Saga se propõe a resolver.

1.4 Aplicação no Projeto

O projeto está estruturado em torno de um serviço central, o orquestrador-service, que intercepta e roteia as mensagens entre o conta-service e o custodia-service, e mantém o registro (TransacaoSaga) de cada transação em processamento. Estruturalmente, essa presença de um coordenador central aproxima o desenho do modelo de Saga Orquestrada, ainda que, na fase atual do projeto, a etapa de decisão sobre compensações — a lógica de estornos — esteja planejada como próxima evolução e ainda não implementada no fluxo de execução.

2. Arquitetura de Microsserviços

2.1 Conceito Geral

Uma arquitetura de microsserviços organiza um sistema como um conjunto de serviços pequenos, independentes e implantáveis separadamente, cada um responsável por uma capacidade de negócio específica. Cada serviço possui seu próprio processo, seu próprio banco de dados e se comunica com os demais por meio de mecanismos leves de rede — chamadas síncronas (REST/HTTP) ou, como neste projeto, mensageria assíncrona. Esse desenho permite que a falha ou indisponibilidade de um serviço não derrube necessariamente os demais.

No projeto, essa filosofia é aplicada por meio de três serviços Spring Boot independentes, cada um em seu próprio módulo Maven, cada um com seu próprio banco H2 e cada um executando em uma porta HTTP distinta:

Serviço	Porta	Papel na arquitetura
conta-service	8081	Porta de entrada do sistema. Recebe a ordem de compra via API REST e publica o evento inicial na fila de mensageria.
orquestrador-service	8083	Coordenador da Saga. Intercepta o evento inicial, registra o estado da transação e a roteia para o próximo serviço.
custodia-service	8082	Último passo do fluxo. Consome o evento roteado pelo orquestrador e persiste o ativo em custódia.

2.2 Serviços do Projeto e as Classes que os Compõem

Cada serviço segue a separação em camadas típica de aplicações Spring Boot: controller (entrada HTTP), service (regra de negócio), domain (entidade JPA), repository (acesso a dados), config (infraestrutura) e, quando aplicável, dto (objeto de transporte entre serviços) e amqp (consumidores de fila).

2.2.1 conta-service

Classe	Camada	Responsabilidade
OrdemCompraController	controller	Expõe o endpoint POST /api/ordens que recebe a ordem de compra em JSON.
OrdemCompraService	service	Persiste a ordem localmente (@Transactional) e aciona o publisher.
OrdemCompraPublisher	service	Publica a ordem no exchange do RabbitMQ.
OrdemCompra	domain	Entidade JPA com id (UUID), cpfCliente, valor e status (nasce como "PENDENTE").

Classe	Camada	Responsabilidade
OrdemCompraRepository	repository	Interface JpaRepository para persistência da entidade OrdemCompra.
RabbitMQConfig	config	Declara fila, exchange (Direct), binding e o conversor de mensagens JSON.

2.2.2 orquestrador-service

Classe	Camada	Responsabilidade
OrquestradorListener	amqp	Consome a fila ordem.compra.fila, cria o registro TransacaoSaga e roteia a ordem para a fila de custódia.
OrquestradorService	service	Consulta o estado de uma Saga por ordemId.
TransacaoSagaController	controller	Expõe GET /api/sagas/{ordemId} para consulta pública do status da transação.
TransacaoSaga	domain	Entidade JPA que registra ordemId, cpfCliente, valor, statusSaga e dataCriacao.
TransacaoSagaRepository	repository	Interface JpaRepository com busca por ordemId.
OrdemCompraDto / RabbitMQConfig	dto / config	DTO de entrada da mensagem e declaração de fila/exchange/binding de custódia.

2.2.3 custodia-service

Classe	Camada	Responsabilidade
CustodiaListener	amqp	Consome a fila custodia.ordem.fila e persiste o ativo recebido.
AtivoCustodia	domain	Entidade JPA com cpfCliente, valor e statusCustodia (definido como "CUSTODIADO").
AtivoCustodiaRepository	repository	Interface JpaRepository para persistência da entidade AtivoCustodia.
OrdemCompraDto / RabbitMQConfig	dto / config	DTO de entrada da mensagem e configuração do conversor JSON (filas/bindings declarados pelo orquestrador).

3. Tecnologias Utilizadas

3.1 Spring Boot

O Spring Boot é o framework base dos três serviços (versão do parent: spring-boot-starter-parent 4.1.0, Java 17). Ele é responsável por:

- Inicializar cada aplicação como um processo independente (classes *Application com @SpringBootApplication).
- Expor endpoints REST via @RestController, @PostMapping e @GetMapping (conta-service e orquestrador-service).
- Mapear entidades para tabelas via Spring Data JPA (@Entity, @Id, @GeneratedValue) e repositórios via JpaRepository.
- Gerenciar a integração com o RabbitMQ através do módulo spring-boot-starter-amqp (@RabbitListener, RabbitTemplate).
- Controlar a atomicidade das operações locais de banco com @Transactional (usado em OrdemCompraService).

3.2 RabbitMQ

O RabbitMQ é o message broker que viabiliza a comunicação assíncrona entre os três serviços, permitindo que o conta-service não precise esperar o processamento completo da Saga para responder ao cliente. O projeto utiliza dois pares de fila/exchange, ambos do tipo Direct Exchange (roteamento exato por routing key):

Exchange	Fila	Routing Key	Produtor → Consumidor
ordem.compra.exchange	ordem.compra.fila	ordem.compra.routing.key	conta-service → orquestrador-service
custodia.ordem.exchange	custodia.ordem.fila	custodia.ordem.routing.key	orquestrador-service → custodia-service

Cada serviço declara sua própria infraestrutura de mensageria (filas, exchanges e bindings) via classes @Configuration com beans do tipo Queue, DirectExchange e Binding — abordagem que evita erros de fila inexistente na inicialização e permite que qualquer serviço suba independentemente da ordem de inicialização dos demais. A conversão de mensagens para JSON é feita pelo JacksonJsonMessageConverter, configurado em todos os três serviços.

3.3 Banco de Dados H2

O H2 é usado como banco de dados relacional em memória (in-memory) em todos os três serviços, com o escopo runtime no Maven. Alguns pontos técnicos importantes sobre seu uso neste projeto:

- **Banco isolado por serviço:** cada serviço tem sua própria instância H2 em memória — orquestradordb (orquestrador-service) e custodiadb (custodia-service) — o que está de acordo com o princípio de banco de dados por serviço da arquitetura de microsserviços.
- **Persistência volátil:** por ser in-memory, todos os dados são perdidos a cada reinicialização do serviço. Isso é adequado para desenvolvimento, testes e demonstração, mas não reflete o comportamento de um banco de produção.

- **ddl-auto=update:** o schema das tabelas é gerado e atualizado automaticamente a partir das entidades JPA, sem necessidade de scripts de migração manuais — prática útil em protótipos, mas que normalmente é substituída por ferramentas como Flyway ou Liquibase em ambientes de produção.
- **Console H2 web (h2-console):** habilitado explicitamente em orquestrador-service e custodia-service (spring.h2.console.enabled=true), acessível em /h2-console em cada porta respectiva, o que permite inspecionar as tabelas transacao_saga e ativo_custodia durante o desenvolvimento.
- **Configuração ausente no conta-service:** o conta-service não define spring.datasource.url nem spring.h2.console.enabled em seu application.properties, de modo que utiliza a configuração padrão de autoconfiguração do Spring Boot para um banco H2 embutido (nome gerado automaticamente) e o console H2 não fica habilitado nesse serviço — uma diferença de configuração em relação aos outros dois, útil de se ter em mente ao inspecionar dados salvos.

3.4 Docker

No estado atual do repositório, o Docker é utilizado exclusivamente para subir a infraestrutura de mensageria, e não para conteinerizar os próprios microsserviços. O README indica o seguinte comando para levantar o RabbitMQ (com o plugin de painel de gerenciamento) em um container:

```
docker run -d --name rabbitmq-server -p 5672:5672 -p 15672:15672 rabbitmq:3-management
```

A porta 5672 é utilizada pelo protocolo AMQP (consumida pelos três serviços Spring Boot) e a porta 15672 expõe a interface web de administração do RabbitMQ (usuário e senha padrão guest/guest). Não há, até o momento, Dockerfile nem docker-compose.yml para os três serviços Java — eles são executados localmente via Maven Wrapper (./mvnw), cada um em seu próprio terminal. Conteinerizar os próprios microsserviços (e orquestrar tudo, incluindo o RabbitMQ, via docker-compose) é um passo natural de evolução da infraestrutura do projeto.

4. Build e Gerenciamento de Dependências (Maven)

O projeto é um build Maven multimódulo: existe um pom.xml raiz do tipo agregador, e cada serviço possui seu próprio pom.xml, com o spring-boot-starter-parent como parent direto (não o pom raiz do projeto).

4.1 pom.xml Raiz (Agregador)

O pom.xml na raiz do repositório tem packaging do tipo pom — ou seja, ele não gera um artefato executável, apenas agrega e organiza os três módulos do projeto, permitindo que um build (mvn install, por exemplo) rodado na raiz processe os três serviços em sequência:

```
<groupId>com.example</groupId>
<artifactId>financial-transactions</artifactId>
<version>1.0</version>
```

```
<packaging>pom</packaging>

<modules>
    <module>conta-service</module>
    <module>custodia-service</module>
    <module>orquestrador-service</module>
</modules>
```

Vale notar que os pom.xml individuais de cada serviço não declaram este pom.xml raiz como <parent> — cada um usa diretamente o spring-boot-starter-parent. Isso significa que o pom raiz cumpre hoje apenas o papel de agregador de módulos (para builds conjuntos), e não o de gerenciador central de versões de dependências entre os serviços.

4.2 pom.xml — conta-service

Parent: spring-boot-starter-parent 4.1.0 · Java 17 · Artifact: conta-service

Dependência	Escopo	Função
spring-boot-starter-amqp	compile	Integração com RabbitMQ (RabbitTemplate, @RabbitListener).
spring-boot-starter-webmvc	compile	Exposição de endpoints REST (Spring MVC).
spring-boot-starter-json	compile	Serialização/desserialização JSON.
spring-boot-starter-data-jpa	compile	Mapeamento objeto-relacional (Hibernate/JPA).
h2	runtime	Banco de dados em memória.
spring-boot-starter-amqp-test	test	Suporte a testes de integração com AMQP.
spring-boot-starter-webmvc-test	test	Suporte a testes de controllers (MockMvc).
spring-boot-starter-data-jpa-test	test	Suporte a testes de repositório (@DataJpaTest).

4.3 pom.xml — custodia-service

Parent: spring-boot-starter-parent 4.1.0 · Java 17 · Artifact: custodia-service

Dependência	Escopo	Função
spring-boot-starter-amqp	compile	Integração com RabbitMQ.
spring-boot-starter-webmvc	compile	Presente no classpath, embora este serviço não exponha controllers REST.
spring-boot-starter-data-jpa	compile	Mapeamento objeto-relacional.
h2	runtime	Banco de dados em memória.
spring-boot-starter-h2console	compile	Habilita o console web do H2 (/h2-console).
spring-boot-starter-amqp-test	test	Suporte a testes de integração com AMQP.

Dependência	Escopo	Função
spring-boot-starter-webmvc-test	test	Suporte a testes de controllers.

4.4 pom.xml — orquestrador-service

Parent: spring-boot-starter-parent 4.1.0 · Java 17 · Artifact: orquestrador-service

Dependência	Escopo	Função
spring-boot-starter-amqp	compile	Integração com RabbitMQ.
spring-boot-starter-webmvc	compile	Exposição do endpoint GET /api/sagas/{ordemId}.
spring-boot-starter-data-jpa	compile	Mapeamento objeto-relacional da entidade TransacaoSaga.
h2	runtime	Banco de dados em memória.
spring-boot-starter-h2console	compile	Habilita o console web do H2 (/h2-console).
spring-boot-starter-amqp-test	test	Suporte a testes de integração com AMQP.
spring-boot-starter-webmvc-test	test	Suporte a testes de controllers.
spring-boot-starter-data-jpa-test	test	Suporte a testes de repositório.

5. Fluxo de Execução Resumido

1. Cliente envia POST /api/ordens (porta 8081) para o conta-service com cpfCliente e valor.
2. OrdemCompraService salva a OrdemCompra (status "PENDENTE") e OrdemCompraPublisher publica o evento em ordem.compra.exchange.
3. OrquestradorListener consome ordem.compra.fila, cria e salva a TransacaoSaga (statusSaga "INICIADA") e reenvia a mensagem para custodia.ordem.exchange.
4. CustodiaListener consome custodia.ordem.fila, cria um AtivoCustodia com statusCustodia "CUSTODIADO" e o persiste no banco custodiadb.
5. O status da transação pode ser consultado a qualquer momento via GET /api/sagas/{ordemId} (porta 8083).

6. Estado Atual e Próxima Evolução

O fluxo descrito acima cobre o caminho de execução com sucesso (happy path) das três etapas da Saga. A próxima etapa planejada para o projeto é a implementação da lógica de estornos (transações compensatórias): o custodia-service publicando um evento de falha, o orquestrador-service atualizando o statusSaga e disparando a compensação, e o conta-service reagindo a esse evento para cancelar/estornar a ordem original, completando, assim, o ciclo que caracteriza o padrão Saga que é achado em artigos, sites e aplicações.